Technical Design Document: TalentMatcher

Author: Ari Choudhary **Status:** Shipped (v1.0) **Last updated:** 2026-03-22

---

1. Problem Statement

A recruiting team has a CSV of 93 sales candidates and multiple open roles. They need to:

1. Rank candidates against a role with explainable, per-criterion evidence
2. Assign candidates optimally across multiple roles simultaneously
3. Control what "good" means (different evaluators have different philosophies)
4. Do this in under 2 minutes, at <$0.15/run, with no ML infrastructure

Existing solutions are either black-box AI ("here's a score, trust us") or manual spreadsheet sorting. Neither provides transparent, configurable, evidence-backed ranking at interactive speed.

2. Design Constraints

Constraint	Impact
Vercel free tier (30s edge timeout)	Scoring must run on a persistent backend, not edge functions
Arbitrary CSV formats	Parser must auto-detect schema; can't require fixed columns
60s scoring latency for 93 candidates	Must stream results, not block on full completion
$0.15 budget per run (free tier viable)	GPT-4o-mini, not GPT-4; embedding pre-filter to reduce call count
No ML infrastructure (no GPUs, no vector DB)	Embeddings via OpenAI API; matching runs client-side
Sparse data (37% have grades, 13% have resume text)	Scoring must degrade gracefully with missing fields

3. Architecture

3.1 System Topology

```
Browser ——— Vercel (Next.js 15, SSR + Edge) ——— App Runner (FastAPI)
                    |                                    |
                  Clerk                          ┌─────────┼─────────┐
                 (Auth)                       DynamoDB  OpenAI    S3
                                             (Sessions) (GPT +   (Photos)
                                                        Embeddings)
```

Why two services? Vercel handles auth, SSR, and static assets. App Runner handles long-running scoring (60s+), the LinkedIn database (5MB in-memory), and Stripe webhooks. The split is forced by Vercel's timeout constraint.

Alternative considered: Vercel Pro (300s timeout). Rejected because it couples scoring logic to the frontend deployment and doesn't solve the in-memory LinkedIn DB problem.

3.2 Component Ownership

Component	Owner	Stateless?	Horizontally Scalable?
Next.js frontend	Vercel	Yes (SSR)	Yes (CDN + edge)
Clerk auth	Managed SaaS	Yes	N/A
FastAPI backend	App Runner	No (LinkedIn DB in memory)	Yes (read-only DB, no writes during scoring)
DynamoDB	AWS managed	N/A	Yes (on-demand)
OpenAI API	External	Yes	Rate-limited (batches of 3)

4. Core Pipeline

4.1 Dual Scoring Modes

```
                        ┌── Rapid Match ──┐
                        │ Structured fields  │
CSV ──→ Parse ──→       │ 8 dimensions       │ ──→ Ranked results
                        │ <1s, $0            │
                        └────────────────────┘

                        ┌── AI Match ──────────────────────┐
                        │ Embed (HyDE) → Top K → GPT judge   │
CSV ──→ Parse ──→       │ 6 criteria, configurable weights   │ ──→ Ranked results
                        │ 18-60s, $0.04-0.14                 │     w/ evidence
                        └────────────────────────────────────┘
```

Rapid Match operates on structured CSV columns (experience years, grades, location, sales focus). It's a weighted point system — no LLM, no API calls, instant. Useful as a first pass or when the CSV has rich structured data.

AI Match operates on unstructured text (concatenated CSV fields + LinkedIn enrichment). It uses GPT-4o-mini as a judge against a configurable rubric. Useful when you need reasoning, evidence, and nuanced evaluation.

They're complementary: Rapid Match is recall-oriented (fast, cheap, broad). AI Match is precision-oriented (slow, costly, deep).

4.2 AI Match Pipeline Detail

```
Input: candidates[], role, criteria[], ideal_candidate, top_k
               │
               ▼
```

```
┌─── Stage 1: Parse (CPU, <1ms/candidate) ───┐
│ Auto-detect name (7-level fallback)          │
│ Auto-detect LinkedIn URL (search all fields)  │
│ Concatenate all fields → fullText             │
└──────────────────────────────────────────────┘
                    │
                    ▼
┌─── Stage 2: Enrich (Network, <1ms/candidate) ──┐
│ Match LinkedIn URL → 493-profile cached DB       │
│ Fallback: fuzzy name match on URL slugs          │
│ Append: experience, education, skills, photo     │
│ Best-effort: skip on miss, never block           │
└──────────────────────────────────────────────────┘
                    │
                    ▼
┌─── Stage 3: Pre-filter (API, ~$0.001 total) ───┐
│ HyDE: embed ideal_candidate profile              │
│ Embed all candidate fullTexts                    │
│ Dot product similarity → rank                    │
│ Keep top K (user-selected: 10/25/50/100/all)     │
└──────────────────────────────────────────────────┘
                    │ top K candidates only
                    ▼
┌─── Stage 4: Judge (API, ~$0.0015/candidate) ───┐
│ For each candidate (batches of 3):               │
│   System: role + rubric + weights + ideal profile│
│   User: candidate fullText (≤3000 chars)         │
│   Response: { score, reasoning, highlights, gaps }│
│   Retry: 3x with 1s backoff                      │
│   Stream: SSE event per scored candidate         │
└──────────────────────────────────────────────────┘
                    │
                    ▼
┌─── Stage 5: Rank + Persist ────────────────────┐
│ Sort by score descending                         │
│ Bucket into tiers (≥70, 50-69, 30-49, <30)       │
│ Compute stats (mean, stddev, cost, tokens)       │
│ Save to DynamoDB (fallback: localStorage)        │
└──────────────────────────────────────────────────┘
```

4.3 Scaling Model

The pipeline is designed around a **funnel invariant**: each stage is ≥10x cheaper than the next and reduces the candidate pool by 5-10x.

Stage	93 candidates	1,000 candidates	10,000 candidates
Parse	93, <1ms	1,000, <10ms	10,000, <100ms
Enrich	93, <100ms	1,000, <1s	10,000, <10s
Embed	93, ~$0.001	1,000, ~$0.01	10,000, ~$0.10

Judge (top 25)	25, ~$0.04	100, ~$0.15	200, ~$0.30
Total	**~$0.04, 18s**	**~$0.16, 70s**	**~$0.40, 140s**

At 10K candidates, the embedding pre-filter reduces GPT calls from 10,000 ($15, 2hr) to 200 ($0.30, 140s). The funnel makes the system sub-linear in cost and time.

At 100K+ candidates: Add a Stage 0 (Rapid Match hard filters on structured fields) to cut to 10K before embeddings. The funnel pattern extends indefinitely.

4.4 Gale-Shapley Multi-Role Matching

When hiring for N roles simultaneously:

```
N roles × M candidates
        │
        ▼
Score each candidate against each role (N×M GPT calls)
        │
        ▼
Build preference matrices:
  rolePrefs[r] = candidates sorted by score for role r
  candPrefs[c] = roles sorted by score for candidate c
        │
        ▼
Gale-Shapley deferred acceptance:
  Roles propose to candidates in preference order
  Candidates accept/reject based on their ranking
  Repeat until stable (no beneficial swaps exist)
        │
        ▼
Output: role→candidate[] mapping + unmatched list
```

Complexity: O(N×M) for the algorithm itself (<1ms). The cost is dominated by the N×M GPT calls. For 3 roles × 50 candidates with top-25 pre-filter: 75 GPT calls, ~$0.11, ~45s.

Capacity support: Roles can hire multiple candidates. `roleCapacities = [3, 2, 1]` means Sales needs 3, Success needs 2, Marketing needs 1. A role stays in the proposal loop until all seats are filled.

5. Data Model

5.1 Candidate (Runtime)

```typescript
interface Candidate {
  id: string;          // "candidate-{index}"
  name: string;        // best-effort from 7-level fallback
  fullText: string;    // ALL CSV fields concatenated
  linkedinUrl: string; // normalized URL or empty
}
```

Design decision: `fullText` is a deliberate lossy concatenation. Individual fields are not preserved. This is intentional — GPT receives everything and decides what matters. Structured field access is Rapid Match's

job.

5.2 Scored Candidate (Runtime)

```
interface ScoredCandidate {
  id: string;
  name: string;
  score: number;           // 0-100
  reasoning: string;       // prose explanation
  highlights: string[];    // strengths
  gaps: string[];          // weaknesses
  evidence: Record<string, string>;  // per-criterion evidence
  criteria: { name: string; score: number; max: number }[];
  embeddingSimilarity?: number;  // HyDE dot product
  tokens: { prompt: number; completion: number };
  cost: number;            // dollars
  photoUrl?: string;
  linkedinUrl?: string;
}
```

5.3 Session (Persisted)

```
interface Session {
  id: string;              // UUID
  role: string;
  role_category: string;
  description: string;
  file_name: string;
  candidate_count: number;
  top_tier: number;        // score ≥ 70
  good_fit: number;        // 50-69
  avg_score: number;
  results: ScoredCandidate[];
  duration: number;        // seconds
  user_id: string;         // Clerk user ID
  user_name: string;
  device: string;          // "desktop" | "mobile"
  tokens: number;
  cost: number;
  judge: string;           // "John" | "Jake" | etc.
  created_at: string;      // ISO timestamp
}
```

DynamoDB schema: PK= `session_id` , SK= `created_at` , GSI on `user_id` . Results stored as JSON string. On-demand capacity.

5.4 Subscription (Persisted)

```
interface Subscription {
  user_id: string;          // PK
  plan: "free" | "pro" | "enterprise";
  postings_used: number;
  postings_limit: number;   // 3, 25, or unlimited
  stripe_customer_id?: string;
}
```

6. SSE Protocol

Scoring uses Server-Sent Events (not WebSockets) because:

1. Unidirectional (server→client only)
2. Native Vercel/App Runner support (no infra changes)
3. Auto-reconnect in EventSource API
4. Simpler than WebSocket lifecycle

Event Types

```
data: {"type":"start","total":25}
data: {"type":"enriched","count":47}
data: {"type":"log","index":0,"name":"Jane Smith","step":"parse","detail":"5 fields"}
data: {"type":"log","index":0,"name":"Jane Smith","step":"enrich","detail":"LinkedIn
match"}
data: {"type":"log","index":0,"name":"Jane Smith","step":"score","detail":"1523 chars
→ GPT"}
data: {"type":"scored","index":0,"id":"c-0","name":"Jane
Smith","score":78,"reasoning":"...","highlights":[...],"gaps":[...],"tokens":
{"prompt":650,"completion":120},"cost":0.000156}
data: {"type":"done"}
```

Frontend contract: Events arrive out of order (batch parallelism). Frontend sorts by score on each `scored` event. `total` from `start` may differ from candidates sent (top-K pre-filter reduces count server-side).

Failure handling: If stream drops, results received so far are preserved in React Context. Session saves with partial results. Duration is tracked from stream start.

7. Rubric System

7.1 The Problem with Black-Box Scoring

An LLM that scores candidates using its own judgment is:

- **Unauditable** — why did candidate A score higher than B?
- **Unrepeatable** — different prompt phrasing → different scores
- **Uncontrollable** — the recruiter's priorities aren't reflected

7.2 Solution: Weighted Criteria + Named Presets

Six criteria with recruiter-adjustable weights:

Criterion	What It Captures	Weight Range

Relevant Experience	Resume fit for the role	0-30%
Industry Fit	Vertical/domain knowledge	0-30%
Sales Capability	Revenue track record	0-35%
Stakeholder Presence	Executive selling ability	0-20%
Cultural Fit	Values, ambition, coachability	0-40%
Location	Geographic fit	0-10%

Five named presets ("judges") map recruiter intuition to weight configurations:

Judge	Philosophy	Highest Weight
John	Balanced generalist	Even distribution
Jake	Pipeline builder	Sales Capability (35%)
Christian	Deal closer	Sales Capability (30%) + Stakeholder (20%)
Yash	Brand-name pedigree	Relevant Experience (30%)
Nazar	Startup builder	Cultural Fit (40%)

Why named judges? Recruiters don't think "set Cultural Fit to 40%." They think "I want someone scrappy." Named presets translate intuition into configuration. Custom mode still available for power users.

7.3 Ideal Candidate Profile

Each judge has an ideal candidate description (prose). This text:

1. Feeds into the GPT system prompt ("evaluate against this profile")
2. Serves as the HyDE embedding document (pre-filter)

Both uses improve with better descriptions. The pill builder UI surfaces options recruiters didn't know to type (e.g., "quota attainment > 120%", "founding team experience").

8. Reliability

Failure Mode	Mitigation
GPT call fails	3x retry, 1s backoff. After 3 failures, skip candidate with score=0
LinkedIn DB timeout	8s timeout. On failure, score without enrichment
DynamoDB write fails	localStorage fallback (max 50 sessions, FIFO eviction)
SSE stream drops	Frontend preserves results received so far
Stripe webhook fails	Idempotent: re-process on next event
OpenAI rate limit	Batch size 3 (concurrent). Backpressure via sequential batches

No single external failure blocks the core pipeline. GPT failure → skip candidate. LinkedIn failure → score without enrichment. DynamoDB failure → save locally. The product degrades gracefully, never crashes.

9. Security

Concern	Mitigation
API key exposure	Server env vars only. Client key stored in localStorage, never transmitted except to OpenAI
Route protection	Clerk middleware: all dashboard routes require auth
Stripe secrets	Server-side only. Webhook signature verification
CSV injection	Input sanitized before GPT prompt (truncated to 3000 chars)
Session access	Sessions scoped to user_id via DynamoDB GSI
LinkedIn DB count leak	Fixed: removed profile count from API response (commit `8773935`)

10. Cost Model

Per-Run Costs (93 candidates)

Component	Top 10	Top 25	Top 50	All 93
Embeddings	$0.001	$0.001	$0.001	$0.001
GPT-4o-mini	$0.015	$0.038	$0.075	$0.140
DynamoDB write	~$0.000001	~$0.000001	~$0.000001	~$0.000001
Total	**$0.016**	**$0.039**	**$0.076**	**$0.141**

Platform Economics

Plan	Price	Postings/mo	Max cost/mo (top 25)	Margin
Free	$0	3	$0.12	-$0.12 (acquisition cost)
Pro	$49	25	$0.98	$48.02 (98%)
Enterprise	Custom	Unlimited	Variable	>90%

The HyDE pre-filter is what makes the free tier viable. Without it, 3 full runs cost $0.42 — still cheap, but 3.5x higher.

11. Testing

Layer	Tool	Coverage
CSV parser	Vitest	Unit: quoted fields, embedded newlines, name detection, LinkedIn URLs

Rapid Match scoring	Manual	93-candidate CSV, verified tier distribution
AI Match scoring	Manual	93-candidate CSV, compared judge rankings
Stable matching	Manual	3 roles × 30 candidates, verified stability property
Auth	Manual	Sign-up flow, protected routes, session persistence
Payments	Stripe test mode	Checkout → webhook → subscription update

Gap: No integration tests for the SSE pipeline. No load testing. No E2E browser tests. These are v2 priorities.

12. Known Limitations

Limitation	Impact	Mitigation Path
LinkedIn DB is 493 static profiles	~50% enrichment rate	Live scraping or LinkedIn API v2
No column mapping UI	Users can't fix auto-detection errors	Manual column assignment dialog
GPT scores are absolute, not relative	Two candidates can score 74 for different reasons	Calibration: score a synthetic baseline candidate first
Weights don't validate to 100%	Frontend allows invalid rubrics	Client-side validation + normalization
No ATS integration	Export is CSV/JSON only	Workable/Greenhouse/Lever API connectors
Single-tenant sessions	No team/org sharing	Add org_id to session schema, ACL layer

13. Future: What a v2 Looks Like

13.1 Calibrated Scoring

Score 3 synthetic candidates (strong/medium/weak) before the real batch. Use their scores to anchor the scale. This converts absolute GPT scores to relative rankings within a calibrated range.

13.2 Active Learning Loop

After a recruiter shortlists candidates, use their accept/reject decisions as training signal. Fine-tune the embedding model on (ideal_profile, accepted_candidate) pairs. Over time, the pre-filter learns what each recruiter actually values — not just what the rubric says.

13.3 Multi-Stage LLM Judge

For 10K+ candidates:

```
10K → Rapid filter → 2K → Embeddings → 200 → GPT-mini → 50 → GPT-4o → Final 25
```

Two-tier LLM judge: mini for bulk screening, full model for deep re-ranking of the shortlist.

13.4 Real-Time Collaboration

WebSocket upgrade for team features: multiple recruiters viewing the same scoring run, live shortlist annotations, shared session commenting.

This document describes the system as shipped. Architecture decisions are explained in context — see the Obsidian vault (`docs/obsidian-vault/`) for deeper exploration of individual subsystems.